

Uncensored AI + REA

The complete setup guide

Abliteration AI + DeepSeek Harness + REA + Ghidra

Get the stack from the video running on your Mac, connect your model, and check that the tools actually work.

Mark Kashef | 12 October 2026 | macOS on Apple silicon

Start with this guide. If you'd rather have your coding agent handle setup, give it the included `AGENT-SETUP-PROMPT.md` and this guide. You'll enter your own API key privately.

Inside the download

- This PDF and its editable Markdown source.
- A copy-ready prompt for your coding agent.
- Configuration examples for the recorded 0.1.x setup and the current 0.2.x release.
- Ghidra environment settings, first-run prompts and troubleshooting.

The software runs locally. The Abliteration AI model runs in the cloud, so model requests leave your Mac. API usage is billed separately by your provider.

01 / What you're setting up

DeepSeek Harness is the workbench: it runs the agent and gives it tools. Abliteration AI supplies the model. REA organizes the reverse-engineering tools, and Ghidra examines compiled programs.

In the recorded run, the agent called REA through terminal commands. You can reproduce that route without registering a new MCP server in all your other coding apps.

Match the video accurately

The recording used DeepSeek Harness **0.1.1-rc.2**, REA **6.4.0**, and Ghidra **12.1.4**. The provider was Abliteration AI, with `abliterated-model-large` selected. Some image requests used `abliterated-model`.

There's a detail worth keeping straight. The dropdown and spoken video call the large model GLM 5.2. The session requested `abliterated-model-large`, but the provider's responses identified `abliterated-model-large-v2`, which the provider describes as GLM 5.3 based. That suggests an alias on the provider's side. The examples preserve the requested model ID; they don't guarantee a fixed backend.

This guide also includes the current **0.2.0-rc.2** setup path. Its configuration location differs from the recorded version. Follow one route throughout.

Before you begin

You need an Apple silicon Mac, Homebrew, Node.js, an Abliteration AI account, and a working folder. REA 6.4.0 accepts Node 22.19+ within 22.x, 24.11+ within 24.x, or stable 26+. It excludes Node 23 and 25.

Check your terminal:

```
uname -m
node --version
brew --version
```

`uname -m` should report `arm64` for this walkthrough. If a command is missing, install it from the official link above before continuing. Intel Macs and Windows need different paths and aren't covered by this walkthrough.

Use software you're authorized to examine. This download contains setup instructions, not Apple's binaries, decompiled files or the Tempo app.

02 / Launch the harness

Current route: 0.2.0-rc.2

Open Terminal and run:

```
mkdir -p ~/re-lab
cd ~/re-lab
npx @deepseek-ai/dsh@0.2.0-rc.2 web
```

Approve npm's package download if prompted. Keep that terminal open. The interface should open at localhost:3080. On the first-run notice, click **Continue**, then **Configure later** when it asks for a DeepSeek key. You're adding Abliteration AI instead. Select `~/re-lab` as the workspace before you send a task.

If the browser doesn't open, visit that address manually. If port 3080 is already in use, stop the old server with Ctrl+C in its own terminal, or add `--port 3081` to the command and use that address instead.

Recorded route: 0.1.1-rc.2

To use the version from the video, substitute this launch command and use the 0.1.x example in section 05:

```
npx @deepseek-ai/dsh@0.1.1-rc.2 web
```

Pinning the version avoids silently moving between configuration formats. These releases are developer previews. The main repository's docs can describe changes newer than either release.

Get your provider key

Create an account and API key in the Abliteration AI console. Check the current pricing and add credit if your account needs it. You don't need a DeepSeek API key for this provider route.

Enter the key directly in the Models page in the next step. Don't paste it into your conversation, a shared screenshot or these example files. A local harness doesn't make a cloud model run locally.

Fastest route with a coding agent

Open `AGENT-SETUP-PROMPT.md` from the ZIP, copy the prompt into your coding agent, and give it this guide. The prompt asks it to inspect your existing setup, preserve your configuration, and report real check results. You'll still need to create your account and enter your own key.

03 / Connect Abliteration AI

In the current Web UI, open **Settings > Models > Add model provider > Custom model API**. Enter:

- **Provider ID:** `abliteration`
- **Display name:** `Abliteration AI`
- **Base URL:** `https://api.abliteration.ai/v1`
- **API protocol:** OpenAI Chat Completions
- **API key:** your own key, entered privately

Under Model catalog, choose **Fetch available models**, select the models you want, then **Add selected** and save the provider. The Provider ID can't be renamed later; the display name can.

If discovery doesn't return a usable list, add the model IDs manually:

- `abliterated-model-large`: the ID requested in the recording; text only, 1,000,000-token context.
- `abliterated-model-large-v2`: an explicit alternative matching the response ID seen in the session; text only, 1,000,000-token context.
- `abliterated-model`: accepts text and images, with a 262,144-token context.

The provider describes the first two as GLM 5.2 based and GLM 5.3 based respectively. That description doesn't resolve the alias observed during recording.

Choose it and check it

Start a new chat. Select **abliteration / abliterated-model-large** from the model picker to match the recorded request. Send a short greeting before starting a long task. This uses a small amount of API credit.

A provider card or dropdown entry alone doesn't prove the connection works. You should get a real reply without an authentication or protocol error. Check the provider console for usage. If you want screenshot input, use the image-capable model in a new chat.

Keep these compatibility settings

The recorded configuration used `supportsDeveloperRole: false` and `maxTokensField: max_tokens`. They control the request format. The UI doesn't expose every compatibility field, so use the example on the next page to preserve these values.

Source: DeepSeek Harness model-provider guide and Abliteration AI model reference.

04 / Current configuration example

For **0.2.0-rc.2**, the standard Web UI profile uses `~/.dsh/profiles/web/cordis.patch.yml`. The Settings header can open the configuration file when your browser is on the same computer.

The full example is `examples/dsh-0.2-cordis.patch.example.yml`. Merge its provider into the existing `llm-pi-ai` entry. Don't replace your entire file: a Cordis override replaces that entry's complete configuration, so retain your other providers and fields.

```
- id: llm-pi-ai
  config:
    providers:
      abliteration:
        displayName: Abliteration AI
        apiKeyEnv: ABLITERATION_API_KEY
        api: openai-completions
        baseURL: https://api.abliteration.ai/v1
        compat:
          supportsDeveloperRole: false
          maxTokensField: max_tokens
        models:
          - id: abliterated-model-large
            contextWindow: 1000000
            input: [text]
          - id: abliterated-model-large-v2
            contextWindow: 1000000
            input: [text]
          - id: abliterated-model
            contextWindow: 262144
            input: [text, image]
```

`apiKeyEnv` is a reference name, not the key itself. If you created the provider through the UI, preserve its saved credential reference. If you're configuring it entirely by file, create `~/.dsh/.env` in a local text editor, add the following line with your real key, and save it privately:

```
ABLITERATION_API_KEY=YOUR_KEY_HERE
```

Restrict that file with `chmod 600 ~/.dsh/.env`. Don't overwrite other existing key lines. Restart the harness after changing `.env`; current profile edits are re-read on the next request. Select the model in a new chat.

05 / The recorded configuration

On the recorded **0.1.1-rc.2** installation, provider settings live in `~/.dsh/settings.yaml`. Use `examples/dsh-0.1-settings.example.yaml` for this route. Don't paste this top-level structure into a 0.2.x profile patch.

This is the core of the recorded request configuration, with credential values excluded:

```
llm-pi-ai:
  providers:
    abliteration:
      displayName: Abliteration AI
      apiKeyEnv: ABLITERATION_API_KEY
      api: openai-completions
      baseURL: https://api.abliteration.ai/v1
      compat:
        supportsDeveloperRole: false
        maxTokensField: max_tokens
      models:
        - id: abliterated-model-large
          contextWindow: 1000000
          input: [text]
        - id: abliterated-model
          contextWindow: 262144
          input: [text, image]
agent-default-model:
  provider: abliteration
  model: abliterated-model-large
```

The downloadable example also lists the explicit `abliterated-model-large-v2` option. Keep the selected default as shown to reproduce the model ID requested in the video.

Merge into existing sections, preserve unrelated settings, and keep your key in the private credential store or `~/.dsh/.env`, as described on the previous page. Restart after editing this older setup, then start a new chat.

If your key change seems to do nothing

The harness checks the launch environment first, then its managed `.credentials.yaml`, then a `.env` in the launch folder, then `~/.dsh/.env`. A key stored through the Models page can take precedence over one you put in `.env`. Update the place that supplies the active credential instead of scattering copies into several files.

06 / Install REA and Ghidra

In Terminal, install the REA release used in the video:

```
npm install --global rea-agents@6.4.0
rea --version
brew install ghidra
```

Homebrew's Ghidra formula includes its Java dependency. On the checked machine these were Ghidra **12.1.4** and OpenJDK **21**. Homebrew can offer newer versions later, so inspect what it installs.

Point REA at those tools in the same terminal:

```
export GHIDRA_INSTALL_DIR="$(brew --prefix ghidra)/libexec"
export JAVA_HOME="$(brew --prefix openjdk@21)/libexec/openjdk.jdk/Contents/Home"
export REA_GHIDRA_STARTUP_TIMEOUT_MS=600000
rea doctor --provider ghidra --json
```

The same lines are in `examples/zshrc-rea.example.sh`. To keep them for future terminal sessions, add them once to `~/.zshrc`. Restart the harness from a fresh terminal afterward so its tool processes inherit the variables.

What success looks like

The scoped doctor's result should show `healthy: true`. Check its Ghidra installation, headless launcher, native decompiler and Java entries. A failing check needs its stated repair; an agent saying “done” doesn't replace the output.

The full `rea doctor` audits other engines and clients too. It can fail for an unrelated integration even when the Ghidra scope passes. Use the scoped command above for this route.

Why this doesn't need `rea setup`

REA is available as both a command-line tool and an MCP server. The recording used the command-line route. REA 6.4.0's setup command doesn't include the newer DeepSeek Harness client integration documented on the repository's main branch. Installing the CLI is enough for an agent with a terminal tool to call it.

Don't register REA with all your other agents just to follow this guide. If you later want an MCP integration, follow that released version's client instructions separately.

07 / Give the agent a first task

Choose a dedicated workspace and keep the permission setting that asks before running actions. On the recorded release, `/permissionPresets` reports the available presets and `/permissionPresets workspace-write` selects the approval-based preset. In a newer UI, use its permission selector; don't guess a renamed command.

In the video I used full access. That's a description of that run, not a requirement for connecting this stack. A workspace folder keeps work organized; it isn't isolation from the rest of your Mac.

First, verify tool access

Send this to a new chat with the Abliteration provider selected:

> From this workspace, run `rea --version` and `rea doctor --provider ghidra --json` using your terminal tool. Show the commands, exit codes and the relevant checks. Don't inspect another app yet. If something fails, explain the failure and proposed repair before changing anything.

If there's no terminal tool or it can't find `rea`, stop there and fix that connection. A successful text reply isn't a tool-execution test.

Then, give it a scoped job

The original opening prompt asked the agent to install REA and help reverse engineer an app, with the REA repository link. It initially stopped at a research report. State the build outcome explicitly:

> Use REA at https://github.com/morluto/rea to examine [PATH TO SOFTWARE I AM AUTHORIZED TO EXAMINE]. Focus on [ONE FEATURE]. Explain what you can establish from actual tool output, then build an original implementation of that feature in this workspace with [MY REQUESTED IMPROVEMENT]. Don't copy the original app's code or assets. Propose the steps first, keep changes inside this project, and show how you tested the result.

Replace the brackets before sending. Start with software you made or an open-source project whose license permits your intended use.

Iterate on something you can see

Open the result. Try the feature. Give a concrete correction: “The speed slider should remember its value for each file,” for example. Ask for a demonstration or test of each claimed fix.

The video is one build, with follow-up work. It doesn't guarantee a one-shot clone, a fixed price or a completion time for your app.

08 / Check the result

The most useful habit from this project is simple: read the evidence behind “done.”

Check the provider. The chosen provider and model ID should match your intent. If logs expose a different response model, record it rather than assuming the dropdown identifies the backend.

Check the tools. A health check must actually pass. If the agent says another tool helped, ask for the command and result. A similarly named package isn't proof it ran the product you meant.

Check the app. Launch it yourself. Test the feature you asked for, including a failure case. The large model is text only; it can't visually inspect your screenshot. Use the image-capable model when you need that feedback.

Check the bill. Use the provider's usage dashboard. An agent's estimate can be wrong, and your run may take more turns than mine. Start with a small task before committing to a long build.

A short acceptance checklist

- The harness opens and uses the intended workspace.
- The selected provider returns a reply in a new chat.
- The agent actually runs its terminal tool.
- `rea --version` reports the intended release.
- `rea doctor --provider ghidra --json` passes its scope.
- Your first task produces the requested report or original implementation.
- You inspect the result and its actual usage cost.

What was checked for this resource

The guide was recovered from the original working session and reconciled with the edited recording's setup-guide promise. Its model IDs were checked against the session provenance and the provider's docs. The configuration examples were parsed, and the local Ghidra scope was checked on the existing Mac installation.

The accompanying `VERIFICATION.md` records the completed smoke checks and their limits. It distinguishes a server starting from a paid provider round trip. This isn't a claim that a fresh Mac installation or every possible target app was tested.

09 / When something doesn't work

The provider rejects every request

Check the base URL, protocol and active credential source. Preserve the two compatibility fields in the example. A 401 usually points to credentials; a 400 needs its exact response message. Don't paste the key into a support conversation.

The model isn't in the picker

Save the provider, confirm the model ID, and start a new chat. In current versions the picker saves the default for new sessions; an existing session can retain its earlier model. If model discovery fails, add the ID by hand.

An image produces a 400 error

The large models are text only. Start a new chat with `abliterated-model`, enable its image input, and attach the screenshot there. An image left in the old session can keep causing the same error.

Ghidra is missing, or Java isn't found

Check the two exported paths, open a fresh terminal, and run the scoped doctor again. Restart the harness from that terminal too. A variable present only in a different terminal won't reach its tools.

Analysis stalls or the app has several architectures

The recorded run hit a universal Mach-O binary that needed a single-architecture working copy. Let the agent inspect the available architectures and the exact REA error before choosing a slice. Don't blindly assume every Mac binary has an `arm64e` slice.

For a diagnosed Objective-C analysis stall, REA supports disabling the `Objective-C Message Analyzer` through `REA_GHIDRA_DISABLED_ANALYZERS`. This can omit analysis; use it only when the error warrants it. Keep any working copies private.

A long lookup keeps restarting

Ask the agent to check the installed CLI's help for snapshot reuse rather than repeatedly launching the same full analysis. Different commands can have different options.

Stop or undo setup

Ctrl+C stops the harness server in its terminal. To remove a provider, use the Models page or restore the configuration backup made before setup. Revoke its API key in the provider console if you no longer need it. Don't delete the whole harness home to remove one provider: it may hold other sessions and settings.

10 / Links and optional routes

Core links

- DeepSeek Harness repository
- Configure models
- Abliteration AI console
- Abliteration AI quickstart
- Abliteration AI models
- Abliteration AI pricing
- REA repository
- REA installation and scoped checks
- Ghidra repository
- Homebrew and Node.js downloads

OrcaRouter

The video also mentions OrcaRouter. It's an optional provider route, not needed for this guide's main setup. Use its official quickstart to obtain the current endpoint and eligible model IDs, then add a separate custom provider. Don't assume every model is unrestricted or available to every account.

Local models

A local model is another option if your computer can run it. LM Studio can expose a local API; use its current server settings and model ID when adding a separate provider. Hardware requirements, tool support and context limits depend on the model. This is an alternative, not the cloud setup used for the recorded build.

Read first, then install

The PDF is usable without installing anything. For assisted setup, the standalone prompt and editable guide are both in the ZIP. `START-HERE.md` maps every file and tells you which configuration example belongs to your version.

Prepared 12 October 2026. Model aliases, prices and interface labels can change. Follow the linked version documentation if your installation differs.

Independent educational resource by Mark Kashef. Product names belong to their respective owners. No affiliation is implied.